OPERATING SYSTEM SIMULATOR

A THIRD-YEAR PROJECT REPORT

OF
OPERATING SYSTEM
(COMP307)

THE DEGREE OF B.Sc. IN COMPUTATIONAL MATHEMATICS

SCHOOL OF SCIENCE
KATHMANDU UNIVERSITY
DHULIKHEL, NEPAL

JULY 2026

SUBMITTED BY

- **Nirjara Ranjitkar** (Group:CM, Roll No:16 , Reg.no: 034904-23)
- **Lakki Thapa** (Group:CM, Roll No:21, Reg.no: 034911-23)
- **Shruti Bhattarai** (Group:CM, Roll No:6, Reg.no 034891-23)
- **Bijika Amatya** (Group:CM, Roll No: 3, Reg.no: 034888-23)

Contents

1 Introduction **3**

2 Objectives **3**

3 Algorithms Used **4**
3.1 CPU Scheduling Algorithm 4
3.1.1 First Come First Serve (FCFS) 4
3.1.2 Shortest Job First (SJF) Scheduling 4
3.1.3 Priority Scheduling 4
3.1.4 Round Robin Scheduling 4
3.2 Memory Allocation Algorithms 5
3.2.1 First Fit: 5
3.2.2 Best Fit: 5
3.2.3 Worst Fit: 5
3.3 Disk Scheduling Algorithms 5
3.3.1 First-come First-serve(FCFS): 5
3.3.2 Shortest Seek Time First(SSTF) : 5
3.3.3 SCAN : 5
3.3.4 C-SCAN : 5
3.3.5 LOOK : 5
3.3.6 C-LOOK : 6
3.4 Performance Metrics 6

4 Implementation **7**
4.0.1 Programming language and Tools used : 7
4.0.2 Project Structure 7
4.1 User Inputs and Outputs 7

5 Screenshots **8**
5.0.1 Home Screen : 8
5.0.2 Input Parameters Interface : 8
5.0.3 Input Parameters Interface : 9
5.0.4 Input Parameters Interface : 9
5.0.5 Input Parameters Interface : 10
5.0.6 Input Parameters Interface : 10

6 Conclusion **12**
6.1 What We Learned 12
6.2 Challenges Faced 12
6.3 Future Improvements 12

1 Introduction

An operating system is a software that acts as an intermediary between user of a computer and the computer hardware. The purpose of an operating system is to provide an environment in which a user can execute programs. It acts as interface between application and hardware. The function of an operating systems are Resource Management, Process Management (CPU Scheduling), Storage Management (Hard Disk), Memory Management (RAM) and Security and Privacy.

Our project introduces the "Operating System Simulator", a computer program built using Python that lets you easily see how a computer's operating system works. We can type in our own information such as computer tasks, memory blocks, or disk requests and instantly watch how the system handles them through helpful charts and visual displays. The program is split into five main sections: Process Manager, CPU Scheduling, Memory Management, Disk Scheduling, and File Allocation with each part visually demonstrating a different core concept of operating systems.

2 Objectives

Our project's main goals are:

1. Build a visual simulator to demonstrate core operating system concepts like CPU scheduling and memory allocation.
2. Compare different algorithms (like FCFS, SJF, and Round Robin) to see their performance and trade offs.
3. Provide an interactive screen that lets users type in their own custom data for testing.

3 Algorithms Used

3.1 CPU Scheduling Algorithm

It is a way for selecting a process from ready queue and put it in CPU for processing.

3.1.1 First Come First Serve (FCFS)

- Jobs are executed on First Come, First Serve (FCFS) basis.
- It is a non-preemptive, preemptive scheduling algorithm.
- Its implementation is based on FIFO queue.
- Poor in performance as average wait time is high.

3.1.2 Shortest Job First (SJF) Scheduling

- It is a non-preemptive scheduling algorithm.
- It is also known as shortest job next (SJF)
- It is a scheduling policy that selects the waiting process with the smallest execution time to execute next.
- The processor should know in advance how much time process will take.
- Best approach to minimize waiting time.

3.1.3 Priority Scheduling

- In this scheduling Priority is assigned for each process.
- The process with the highest priority is executed first, and so on.
- Processes with the same priority are executed in FCFS manner.
- Priority can be decided based on memory requirements, time requirements, or any other resource requirements.

3.1.4 Round Robin Scheduling

- CPU is assigned to the process for a fixed amount of time.
- This fixed amount of time is called as time quantum or time slice.
- After the time quantum expires, the running process is preempted and sent to the ready queue.
- Then, the processor is assigned to the next arrived process.

- It is always preemptive scheduling algorithm.

3.2 Memory Allocation Algorithms

3.2.1 First Fit:

This algorithm allocates the first hole that is big enough. Searching can start either at the beginning of the set of the holes or at the location where the previous first-fit search ended. We can stop searching as soon as we find a free hole that is large enough.

3.2.2 Best Fit:

Allocates the smallest hole that is big enough. We must search the entire list, unless the list is ordered by size. This strategy produces the smallest leftover hole.

3.2.3 Worst Fit:

Allocates the largest hole. Again, we must search the entire list, unless it is sorted by size. This strategy produces the largest leftover hole, which may be more useful than the smaller leftover hole from a best-fit approach.

3.3 Disk Scheduling Algorithms

3.3.1 First-come First-serve(FCFS):

It is a disk scheduling algorithm that serves I/O requests in the exact order they arrive in the queue. This method is simple to implement and is often inefficient.

3.3.2 Shortest Seek Time First(SSTF) :

It is a disk scheduling algorithm that selects the reqiest closest to the current disk arm position, minimizing seek time. It improves performance compared to FCFS by reducing average response time and increasing system throughput. But this may cause starvation of distant requests.

3.3.3 SCAN :

It is a method where the disk arm moves in one direction, servicing requests along the way, and then reverses direction at the end, similar to an elevator.

It provides better performance than simple algorithms but may cause longer waiting time for some requests.

3.3.4 C-SCAN :

It is an improvement over the SCAN in one direction only. Instead of reversing, it jumps back to the beginning of the disk and continues servicing requests, providing more uniform wait times.

3.3.5 LOOK :

It is similar to SCAN, but instead of moving the disk arm to the end, it only goes up to the last pending request in that direction and then reverses, reducing unnecessary movement.

3.3.6 C-LOOK :

It is similar to C-SCAN but avoids the unnecessary movement by going only up to the last request in one direction and then jumping to the last request at the other end, instead of going to the disk extreme end.

3.4 Performance Metrics

Metric	**Formula**
Waiting Time	Turnaround Time − Burst Time
Turnaround Time	Completion Time − Arrival Time
Seek Time (Disk)	Sum of head movements between consecutive requests

4 Implementation

4.0.1 Programming language and Tools used :

We built this project using python 3 because it is simple to work with and has a built in support for basic GUI framework TKinter.

4.0.2 User Inputs and Outputs

- **Process Manager:** You enter a process name and memory size. The system assigns memory from a simulated space and shows a live map. Ending a process frees the memory and merges it back into the free space.
- **CPU Scheduling:** You add processes with their ID, arrival time, run time, and priority, then pick an algorithm and time limit. The system outputs a timeline chart and a table of waiting and turnaround times.
- **Memory Management:** You define memory blocks, process requests, and pick a method like First Fit, Best Fit, or Worst Fit. The output shows a color-coded diagram with space and usage stats.
- **Disk Scheduling:** You set the disk size, start position, and track requests, then choose an algorithm. The system shows the head movement path, track order, and total seek time.
- **File Allocation:** You create files with a name, size, and content. The system shows how blocks are assigned on a disk using contiguous, linked, or indexed allocation.

5 Screenshots

5.0.1 Home Screen :

This image shows the main dashboard of our OS Algorithm Simulator, an interactive learning tool for exploring core operating system concepts. The given interface organizes essential system modules into clear cards, including Process Manager, CPU Scheduling, Memory Management, and Disk Scheduling. Users can easily navigate through these options to visualize and test foundational computer science algorithms like FCFS, Round Robin, and memory allocation methods.

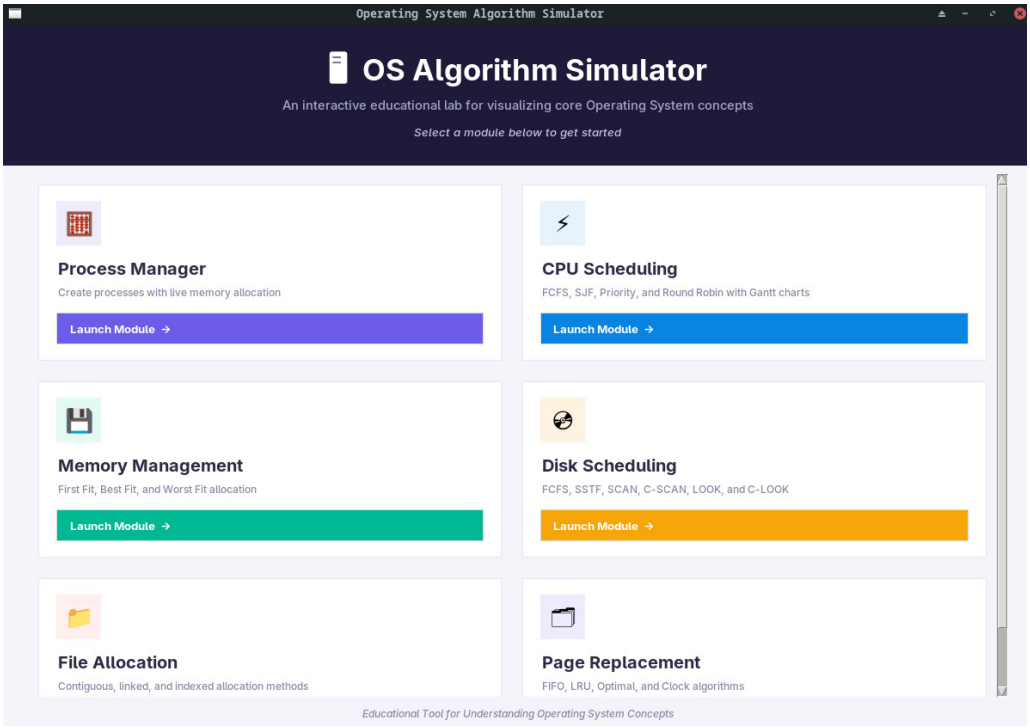

Figure 1: Main Dashboard

5.0.2 Input Parameters Interface :

Process Manager

This image shows the Process Manager - Live Memory Allocation screen of our OS simulator. It features a control panel to create new processes alongside a physical memory map that displays how RAM layout and free space change in real time. A system log at the bottom tracks updates and memory actions as processes run.

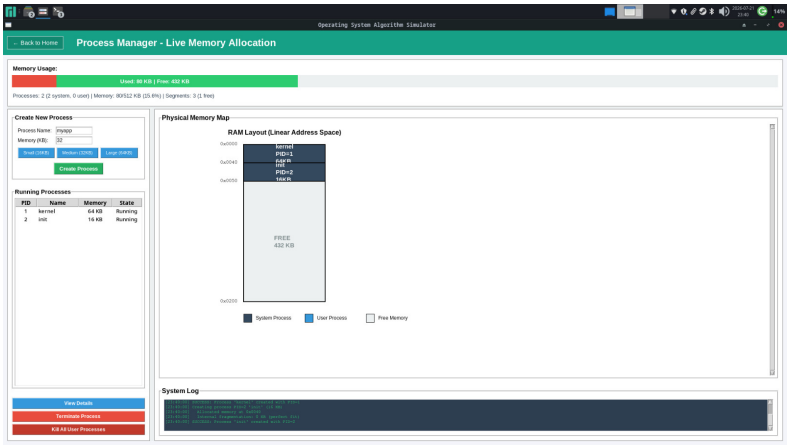

Figure 2: Live Memory Allocation

Memory Management Algorithm :
This image shows the Memory Management Algorithms screen of our OS simulator. It lets users choose between different allocation methods like First Fit, Best Fit, and Worst Fit, while providing controls to add memory blocks and test process requests. A visual layout at the top displays the current state of free and allocated memory blocks in real time.

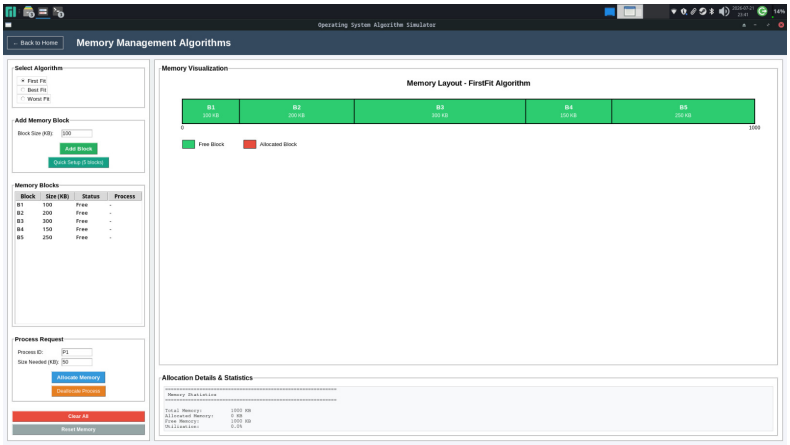

Figure 3: Memory Management Algorithms

File Allocation Methods :
The given image shows the File Allocation Methods screen of the OS simulator. It allows users to switch between allocation strategies like Contiguous, Linked List, and Indexed Allocation, while providing a tool to create and manage files on disk. A visual grid layout at the right displays how storage blocks are used and assigned in real time.

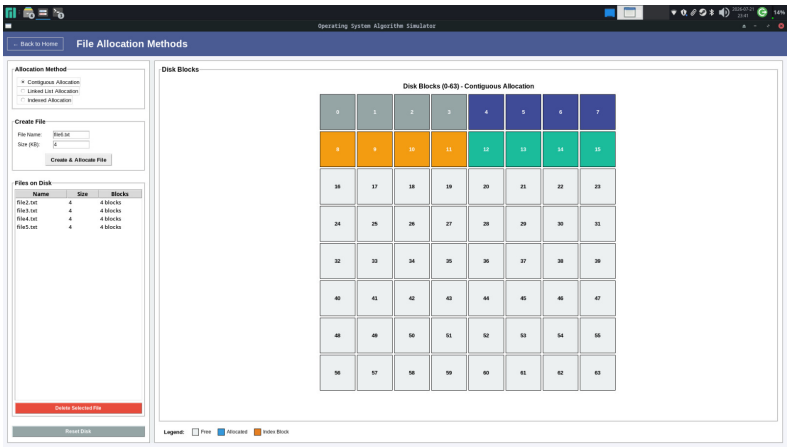

Figure 4: File Allocation Methods

Disk Scheduling Algorithm :
The given image shows the Disk Scheduling Algorithms screen of our OS simulator. It lets users choose from different scheduling methods like FCFS, SSTF, and SCAN, while setting disk parameters and request queues. The

interactive graph on the right visualizes the exact path and movement of the disk head across different tracks.

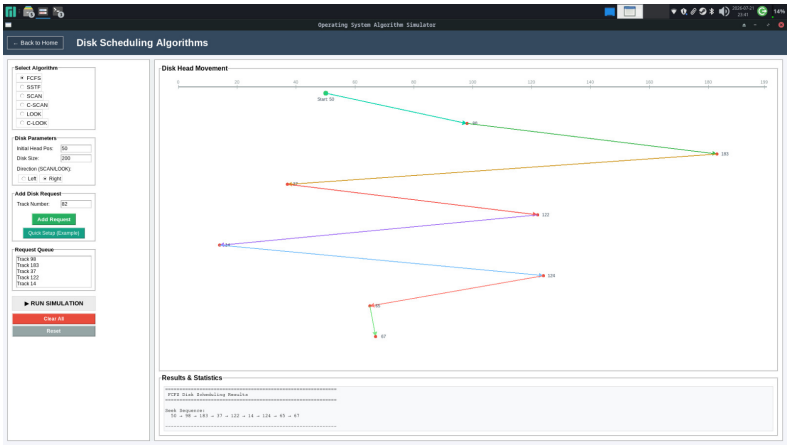

Figure 5: Disk Scheduling Algorithms

Page Replacement Algorithm :
The given image shows the Page Replacement Algorithms screen of our OS simulator. It lets users choose between different strategies like FIFO, LRU, and Optimal to see how an operating system decides which memory pages to remove. A timeline and statistics section at the bottom track performance metrics like page faults, hits, and ratios.

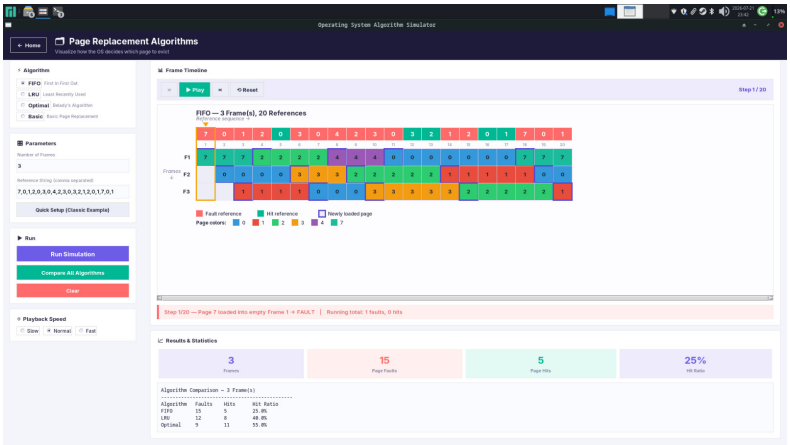

Figure 6: Page Replacement Algorithms

6 Conclusion

6.1 What We Learned

After building this simulator, it helped us understand CPU scheduling, memory allocation, and disk scheduling better by turning theory into working code. After comparing algorithms like FCFS, SJF, Priority, and Round Robin, we learned about their strengths and weaknesses clearly. When we implement memory allocation methods like First Fit, Best Fit, Worst Fit and disk scheduling method like SCAN, C-SCAN, LOOK, C-LOOK,it made fragmentation and seek time easier to understand. We also improved our skills in building a multimodule Tkinter app and separating UI from logic.

6.2 Challenges Faced

- Handling process arrival times in preemptive and non-preemptive algorithms, especially when multiple processes arrive at once.
- Keeping the Round Robin ready queue in the right order when new and requeued processes mix.
- Designing scalable and readable Gantt charts, memory maps, and disk-head paths on canvas.
- Correctly merging free memory after process termination to avoid fragmentation issues.
- Keeping the GUI responsive and code organized across five modules while sharing common features like reset and results display.

6.3 Future Improvements

- Add animation speed control for step by step viewing.
- Support preemptive SJF and Priority scheduling.
- Allow saving and loading custom inputs.
- Increase the process limit beyond 20.
- Add a side-by-side comparison view for all algorithms in a category.